

CRITERION A

The problem

For most of my conscious life I remember myself doing different type of sports and partially or mostly following healthy diet, staying fit. In January 2024 I've decided to count all nutrition I eat to achieve specific body parameters. Counting proteins, fats and hydrocarbons that I've eaten through the day without any automation I found very time consuming. I've looked through different popular food tracing apps but haven't found any that was at the same time easy to get used to, easy to customize, private - not collecting my personal data and cheap or at least, not very expensive.

When discussing fitness and nutrition counting with some of my fitness-conscious friends, they mentioned the problems of privacy and affordability with apps that help to keep nutrition track. I've decided to create my own open-source application, and three of my friends and my grandmother have agreed to be my app testers.

Product

The product is intended as open-source, web-based nutrition tracker that gives users ability to conveniently make a detailed log of their dietary intake. Open source (located on GitHub) makes it product freely modified and main idea is that anyone can freely and continently host his own version of tracker on his own server, with full control over data privacy and system customization. So, the product aims to address my personal needs and all people on the internet, who want to track their nutrition's and who value data privacy and customization, this way product is being an alternative to commercially available options.

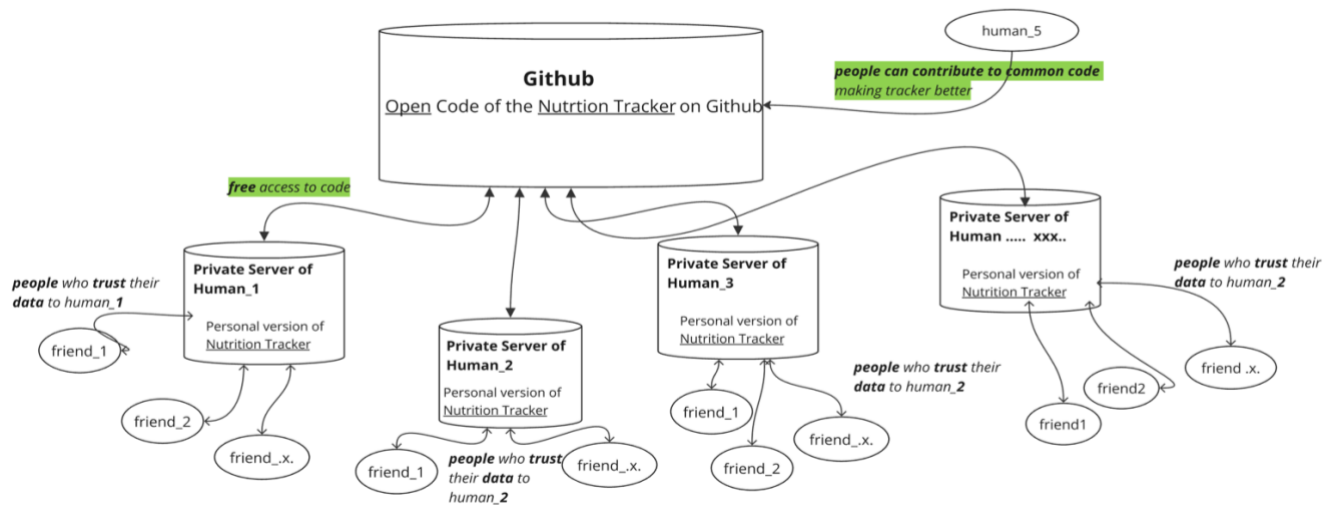
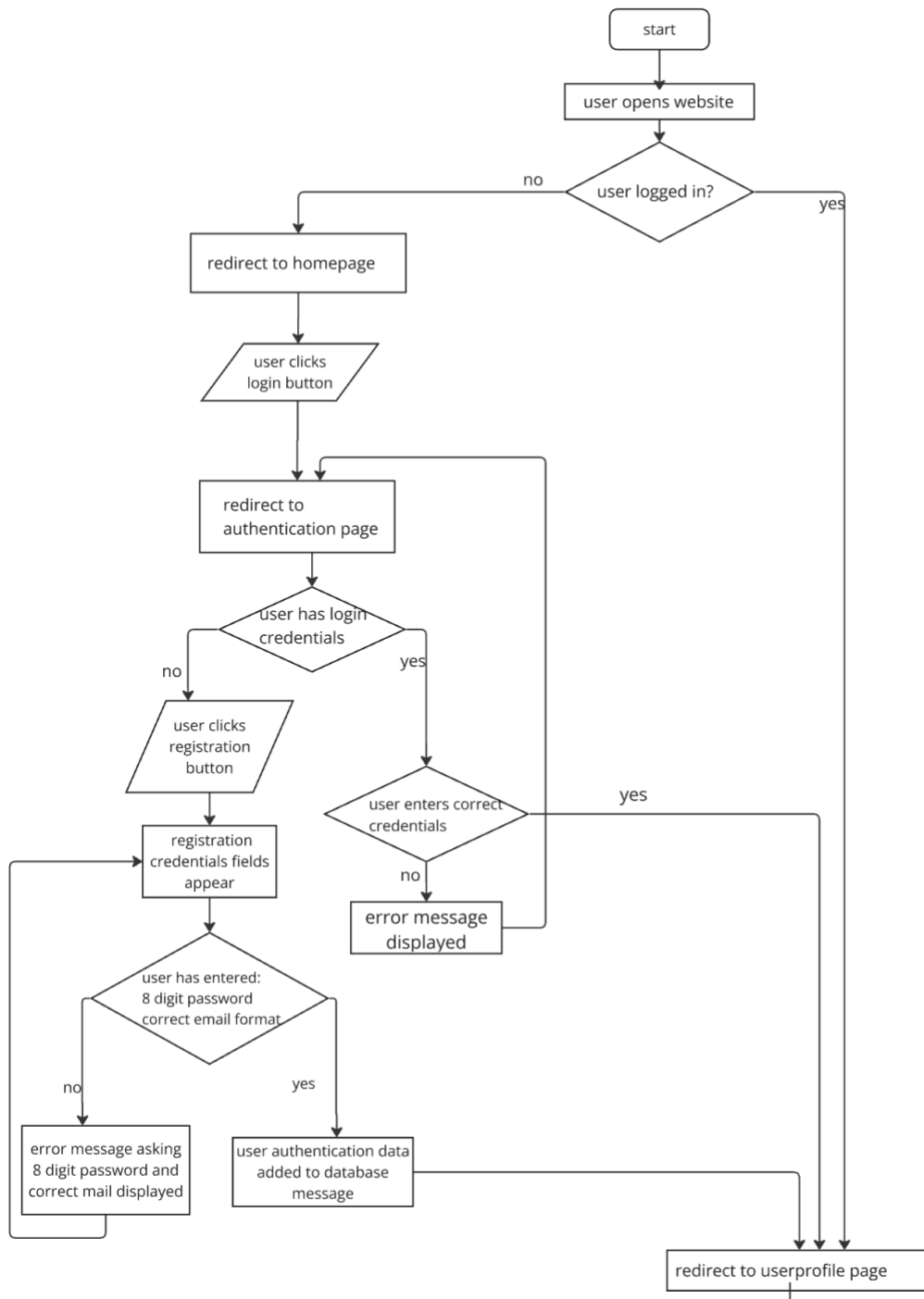


Figure 1. Aimed purpose / application of the product. Created with Miro.

Success criteria

1. Tracking amount of nutrition: (carbs, fats, protein and equivalent in kilocalories) by entering the amount of entered food, in a convenient way.
2. Historical data viewing: users can track their own history of consumption in past weeks, months and year.
3. User authentication system: necessary for securely tracking and accessing individual users' nutrition history, ensuring data privacy and personalized access to each user's records.
4. Data privacy: tracker is open-source application that doesn't send users data anywhere and designed to protect the data.
5. Customizability: users are not limited by existing app database and can conveniently add their own meals with custom parameters to the database.
6. Usability: Instructions and scripts for convenient installation on server, with intuitive and simple interface of application itself.

Word count: 376 words.



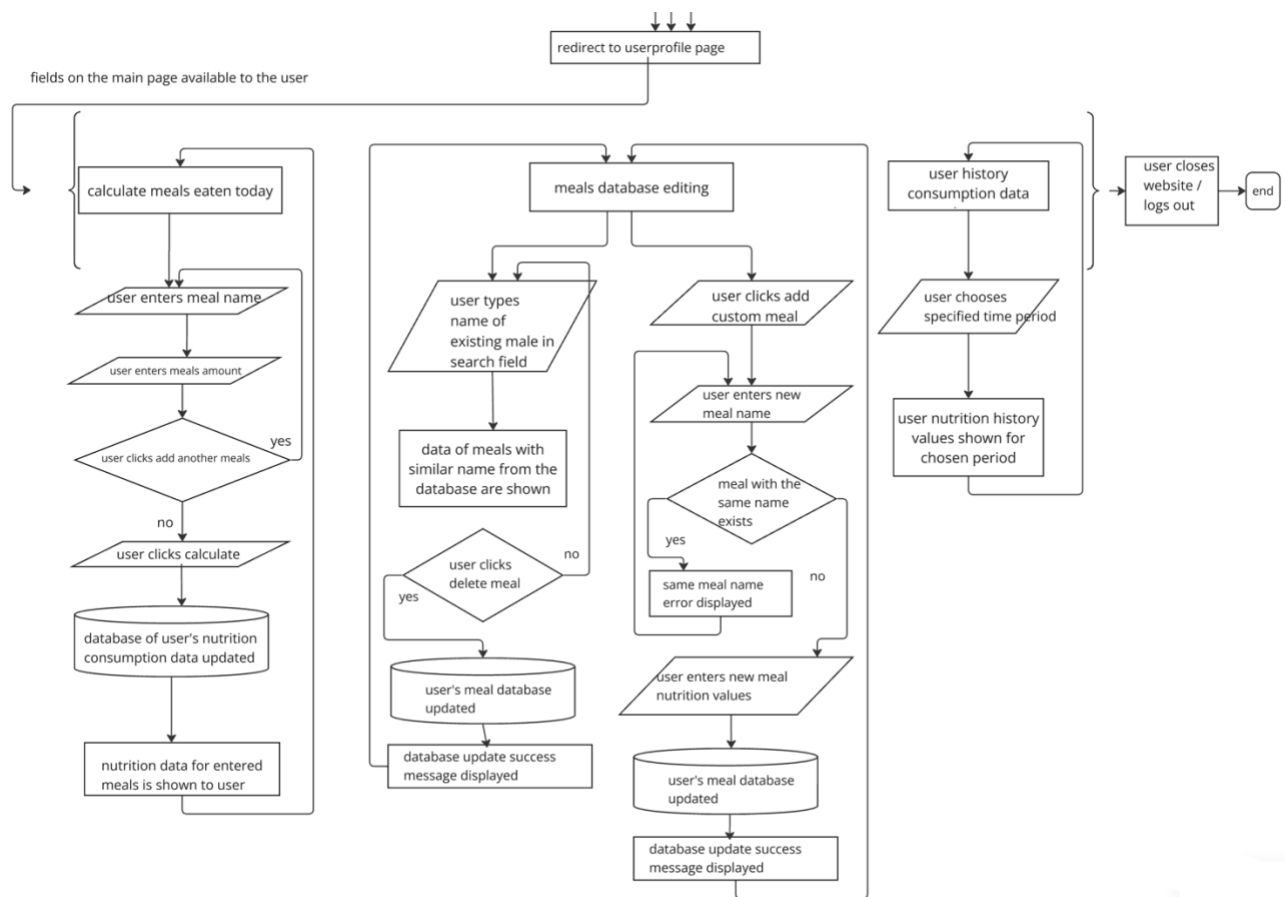


Figure 2. Nutrition Tracker sequence chart flow. Created with Miro.

Test plan

Test type	Nature of test	Example	Expected result
User Registration	Check that the user can register successfully	At registration page, enter correct details, submit	User is created in the database; And logged in
User Login	Verify that users can log in with correct credentials	User enters username and password, submits	User is logged in;
Add Custom Food	Database scheme design	User fills out the form to add a new food item and submits	New food item is added to the database;

CRITERION C

System architecture

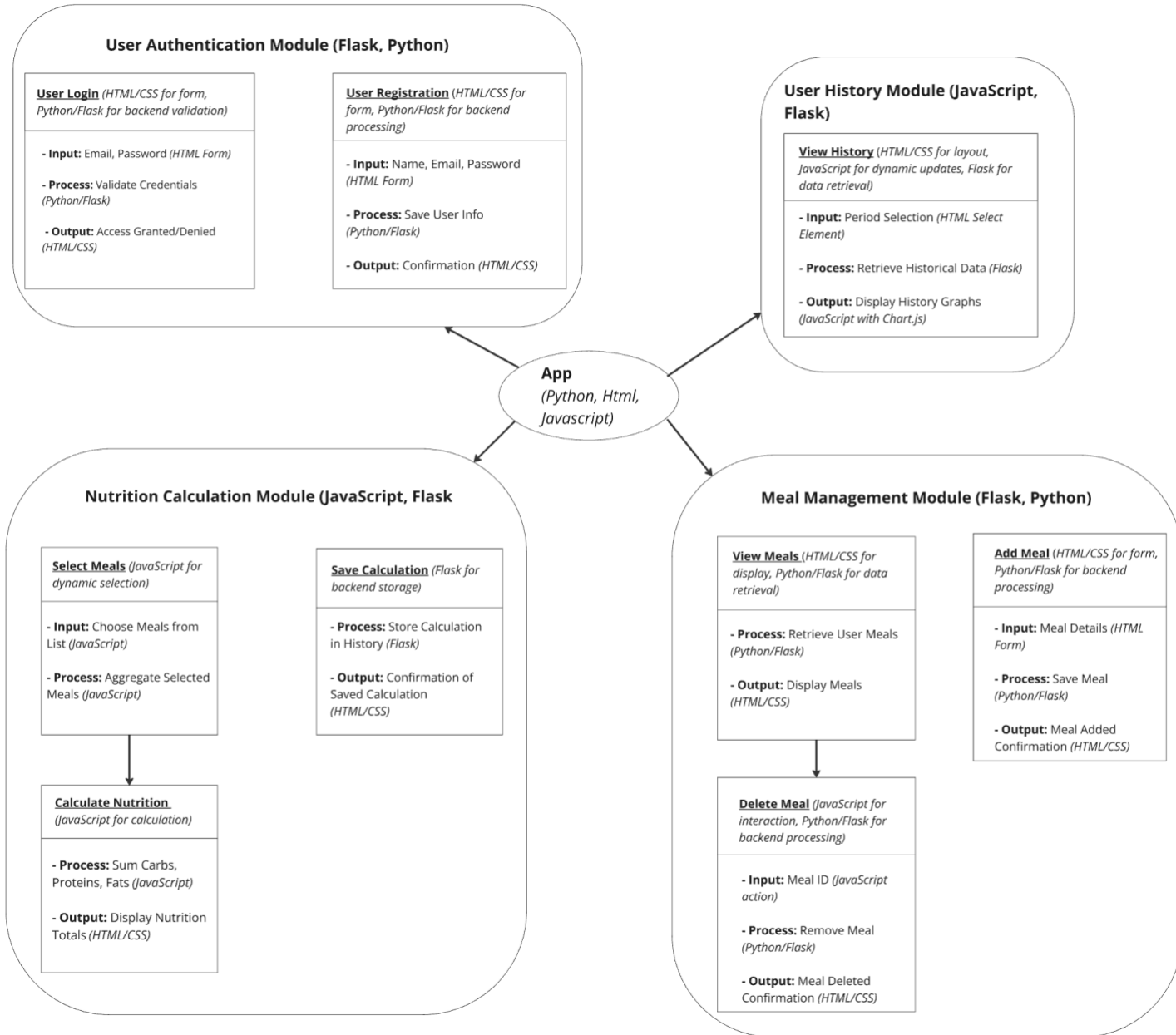


Figure 3. Nutrition Tracker system architecture. Created with Miro.

- b. Displays totals and sends data to the server via a POST request to /calculate.
- c. After calculation, updates nutrition goals chart with latest data.

Additional:

- Chart.js is used for creating a horizontal bar chart to visualize the user's progress in daily nutritional goals.
- Bootstrap provides responsive design and styling for the page.
- jQuery, Popper.js, and Bootstrap.js are used for frontend functionality and interactivity.

Relationships in Database:

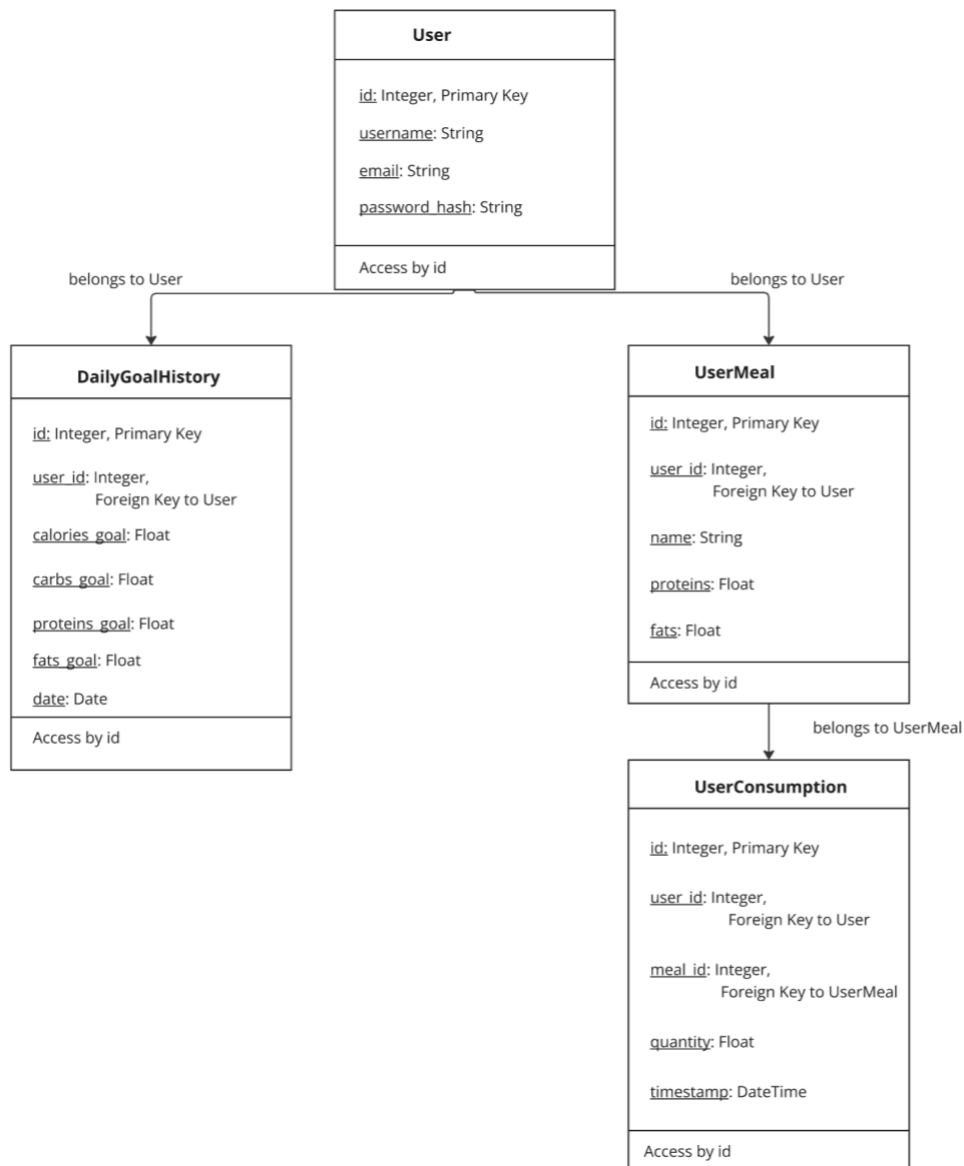


Figure 4. Nutrition Tracker Database Architecture. Created with Miro.

Libraries and Frameworks:

1. Python 3: Python is the primary programming language used for the backend. It was chosen because it is well-known to me as a developer, and it is also true that integrating Python has a variety of libraries for most of the life cases.
2. Flask: Flask is a light web framework allows building web applications efficiently in terms of development time. It allows to have lots of functionality in few lines of code. It was used to create the core structure of the application: handling routing, request processing, session management. This framework was chosen because it is flexible and good known for rapid web development.
3. Flask-Login: Flask-Login manages user authentication and session handling. It enables users to log in securely and track their data across sessions. Obviously, it was chosen because this tool greatly simplifies user session management.
4. SQLAlchemy: SQLAlchemy is an ORM (Object-relational mapping technique) that handles interactions with the database using Python objects. It was used to create and manage the application's database of users and meals. SQLAlchemy was because of its ability to abstract SQL queries, easing code development, readability and security against SQL injection.
5. Jinja2: Jinja2 is a templating engine integrated with Flask for rendering dynamic HTML pages. It was used to generate the front end of the application with dynamic content display. Jinja2 was selected because of integration with Flask and ability to manage dynamic web content.
6. Chart.js: Chart.js is a JavaScript library for creating dynamic charts. It was used to visualize users' nutrition history data, making it easier for users to interpret their dietary intake. This library was chosen because of simplicity and still effectiveness in creating interactive visual data.
7. Bootstrap: Bootstrap is a front-end CSS framework used to style the application. It was applied to make the interface responsive and visually appealing across different devices. Bootstrap was chosen because it provides an easy way to implement a modern design.

8. JavaScript & jQuery: JavaScript with jQuery, was used to handle interactivity and AJAX-based updates, such as submitting forms and fetching data asynchronously. These were chosen to improve user experience by avoiding full page reloads, providing application responsiveness.
9. Flask-WTF & WTForms: Flask-WTF simplifies form creation and validation, making it easier to handle user input. WTForms is used for form field validation, ensuring data integrity. These were chosen because they reduce the amount of code needed for form handling.
10. Werkzeug: is a WSGI utility library used by Flask for routing and request handling. It also provides password hashing functions, which were used to make user passwords stored securely. This library was chosen for its security features and tight integration with Flask.
11. jQuery: jQuery is a JavaScript library used for DOM manipulation and making AJAX requests. It was chosen to simplify client-side scripting and improve user interaction on the frontend side.

TechwithTim's Authentication Module

Since this was my first time developing an authentication system on the website, I've used TechwithTim's Flask Web App Tutorial code on Github as a template for the authentication module (auth.py).

Bibliography

1. RoseHosting Blog. "How to Install Flask on Ubuntu 22.04 with Apache and WSGI." Retrieved from: <https://www.rosehosting.com>
2. TechwithTim GitHub Repository. "Flask Web App Tutorial." Retrieved from: <https://github.com/techwithtim/Flask-Web-App-Tutorial>
3. Flask Documentation. "Flask Web Development." Retrieved from: <https://flask.palletsprojects.com>
4. SQLAlchemy Documentation. "SQLAlchemy ORM Documentation." Retrieved from: <https://docs.sqlalchemy.org>
5. Chart.js Documentation. "Chart.js." Retrieved from: <https://www.chartjs.org/docs/latest>

6. Bootstrap Documentation. "Bootstrap." Retrieved from:
<https://getbootstrap.com/docs>
7. Flask-Login Documentation. "User Session Management for Flask."
Retrieved from: <https://flask-login.readthedocs.io/en/latest>
8. Flask WT Forms Documentation. "Simple Integration of WTForms with Flask." Retrieved from: <https://flask-wtf.readthedocs.io/en/stable>
9. jQuery Documentation. "jQuery API Documentation." Retrieved from:
<https://api.jquery.com>
10. Common meals dataset. U.S. Department of Agriculture. Retrieved from:
<https://fdc.nal.usda.gov/download-datasets.html>

CRITERION E

Success Criteria Evaluation

Criteria	Comments	Achieved / Not achieved
<u>Tracking amount of nutrition</u>	Users can input meals consumed, and the application calculates nutritional intake accurately.	achieved ✓
<u>Historical data viewing</u>	Users can view their nutrition history over selected periods, with data visualized through interactive charts.	achieved ✓
<u>User authentication system</u>	Users can register with unique credentials and securely log in to access personalized features.	achieved ✓

<u>Data privacy</u>	Tracker can be easily self-hosted, as its source code is available on github: https://github.com/naborsimvolov/web_nutrition_tracker_v2	achieved ✓
<u>Customizability</u>	Users can add, edit, and delete custom meals. It was important for testers that some predefined database exists.	achieved ✓
<u>Usability</u>	According to responses of 3 testers interface is intuitive and responsive.	achieved ✓

Client feedback

Myself I started to use nutrition tracker every day since its first working version, so throughout development of nutrition tracker I was able to find most of bugs and fix them. I have provided three of my friends and my grandmother with a link to tracker running on a server in July, when I was in Russia. In a week I was able to collect responses from them, so the first round of feedback gave me the following:

- Having to manually populate database with foods of everyday all users have found inconvenient.
- On calculate today page, after each calculation, nutrition charts randomly changed y-axis position on a screen for users with Safari browsers.
- On meals management page, suggestion list appeared large, taking too much screen.
- All users found design to be simplistic.

After all issues were addressed, I've gained second round of feedback in September:

- My grandmother found design to be dark "associates with death"